Experiment No. 6

Implementation and Comparative Analysis of Image Segmentation Techniques using Python

and OpenCV

In [2]:
```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Step 1: Load image
image = cv2.imread("image.jpg.png")
if image is None:
    print("Image not found!")
    exit()

# Step 2: Grayscale and Gaussian Blur
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Step 3: Global Thresholding
_, global_threshold = cv2.threshold(blur, 127, 255, cv2.THRESH_BINARY)

# Step 4: Otsu's Thresholding
otsu_value, otsu_threshold = cv2.threshold(
    blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print("Otsu Threshold:", otsu_value)

# Step 5: Adaptive Thresholding
adaptive_threshold = cv2.adaptiveThreshold(
    blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)

# Step 6: Watershed Segmentation
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(otsu_threshold, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.7 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

_, markers = cv2.connectedComponents(sure_fg)
markers += 1
markers[unknown == 255] = 0

watershed_image = image.copy()
markers = cv2.watershed(watershed_image, markers)
watershed_image[markers == -1] = [255, 0, 0]

# Step 7: K-Means Clustering
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
pixels = np.float32(rgb.reshape((-1, 3)))

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = kmeans.fit_predict(pixels)
centers = np.uint8(kmeans.cluster_centers_)

segmented_image = centers[labels].reshape(rgb.shape)

# Step 8: Compare Results
plt.figure(figsize=(12, 8))

images = [
    (rgb, "Original Image"),
    (global_threshold, "Global Thresholding"),
    (otsu_threshold, "Otsu Thresholding"),
    (adaptive_threshold, "Adaptive Thresholding"),
    (cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB), "Watershed"),
    (segmented_image, "K-Means")
]

for i, (img, title) in enumerate(images, 1):
    plt.subplot(2, 3, i)
    plt.imshow(img, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()

# Step 9: Observations
print("\nObservations:")
print("Global: Simple and fast")
print("Otsu: Automatic threshold")
print("Adaptive: Handles uneven illumination")
print("Watershed: Separates touching objects")
print("K-Means: Color-based segmentation")

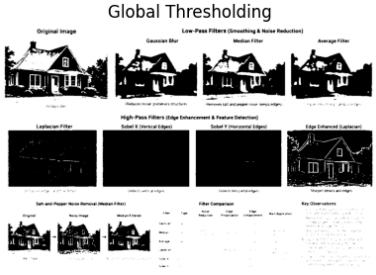
# Step 10: Save Results
cv2.imwrite("global_threshold.jpg", global_threshold)
cv2.imwrite("otsu_threshold.jpg", otsu_threshold)
cv2.imwrite("adaptive_threshold.jpg", adaptive_threshold)
cv2.imwrite("watershed.jpg", watershed_image)
cv2.imwrite("kmeans_segmentation.jpg",
            cv2.cvtColor(segmented_image, cv2.COLOR_RGB2BGR))
```

```
Otsu Threshold: 150.0
```

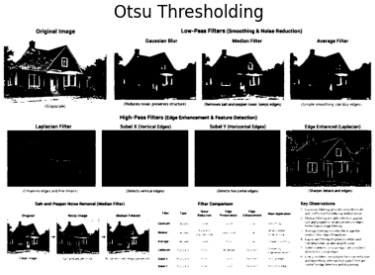
Original Image



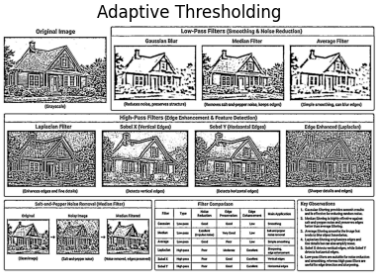
Global Thresholding



Otsu Thresholding



Adaptive Thresholding



Watershed



K-Means



```
Observations:
Global: Simple and fast
Otsu: Automatic threshold
Adaptive: Handles uneven illumination
Watershed: Separates touching objects
K-Means: Color-based segmentation
```

Out[2]: True